

Relatório Técnico

Controlador de Vending Machine em SystemVerilog

Trilha: RTL Design – Módulos: Projeto de Controlador Digital e Verificação de Equivalência Formal

Aluno: Apolo de Lima Silva
Equipe: Individual
Data: 31 de julho de 2026

1. Link para o repositório GitHub do projeto

O projeto foi organizado conforme a estrutura abaixo:

```
grupo_NN_vending/
|-- relatorio_equivalencia.pdf
|-- Makefile
|-- rtl/
| |-- vending_pkg.sv
| |-- credit_reg.sv
| |-- memory.sv
| |-- comparator.sv
| |-- subtractor.sv
| |-- control_unit.sv
| '-- vending_top.sv
|-- sim/
| '-- tb_vending.sv
|-- synth/
| |-- synth.tcl
| |-- vending.sdc
| |-- vending_top_syn.v
| '-- reports/
| |-- default.svf
| '-- ...
'-- fm/
    |-- formality.tcl
    |-- formality_auto.tcl
    '-- reports/
```

O repositório correspondente é:

https://github.com/apoloSilva/grupo_NN_vending

2. Diagrama de blocos

A arquitetura foi implementada por meio de uma unidade de controle (`control_unit.sv`) e de módulos que compõem o caminho de dados (`credit_reg.sv`, `memory.sv`, `comparator.sv` e `subtractor.sv`). Além disso, foi necessário implementar um registrador em `vending_top.sv` para a saída registrada `change_out`. As saídas `display` e `state_out` são registradas pelos respectivos módulos: `comparator.sv` e `control_unit.sv`. A Figura 1 apresenta o diagrama em blocos do sistema desenvolvido, com a identificação, em vermelho, do caminho do sinal de `clock`.

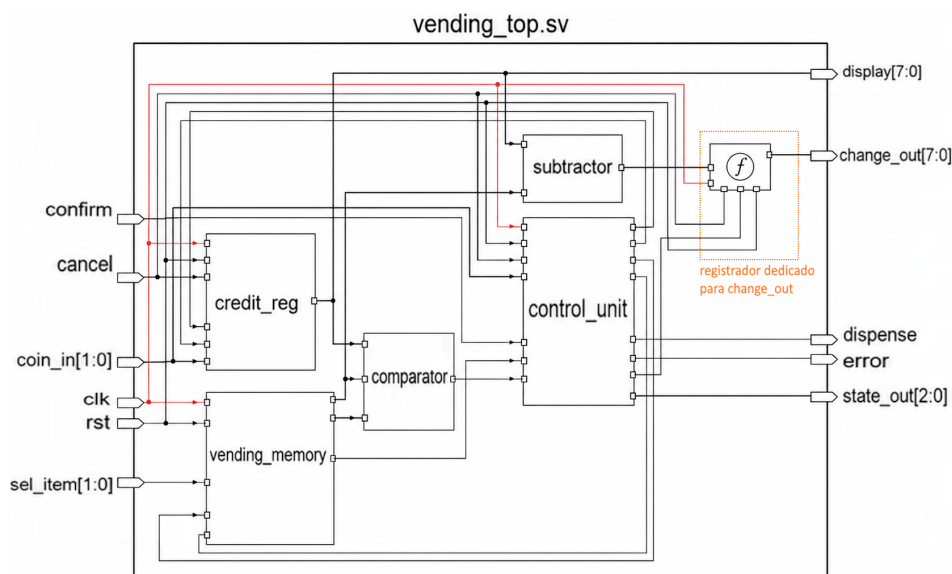


Figura 1: Diagrama geral em blocos do controlador de *vending machine*.

3. Diagrama de estados

A FSM possui seis estados codificados em três bits. A Figura 2 apresenta o diagrama de estados da máquina de estados implementada na unidade de controle, no arquivo `control_unit.sv`.

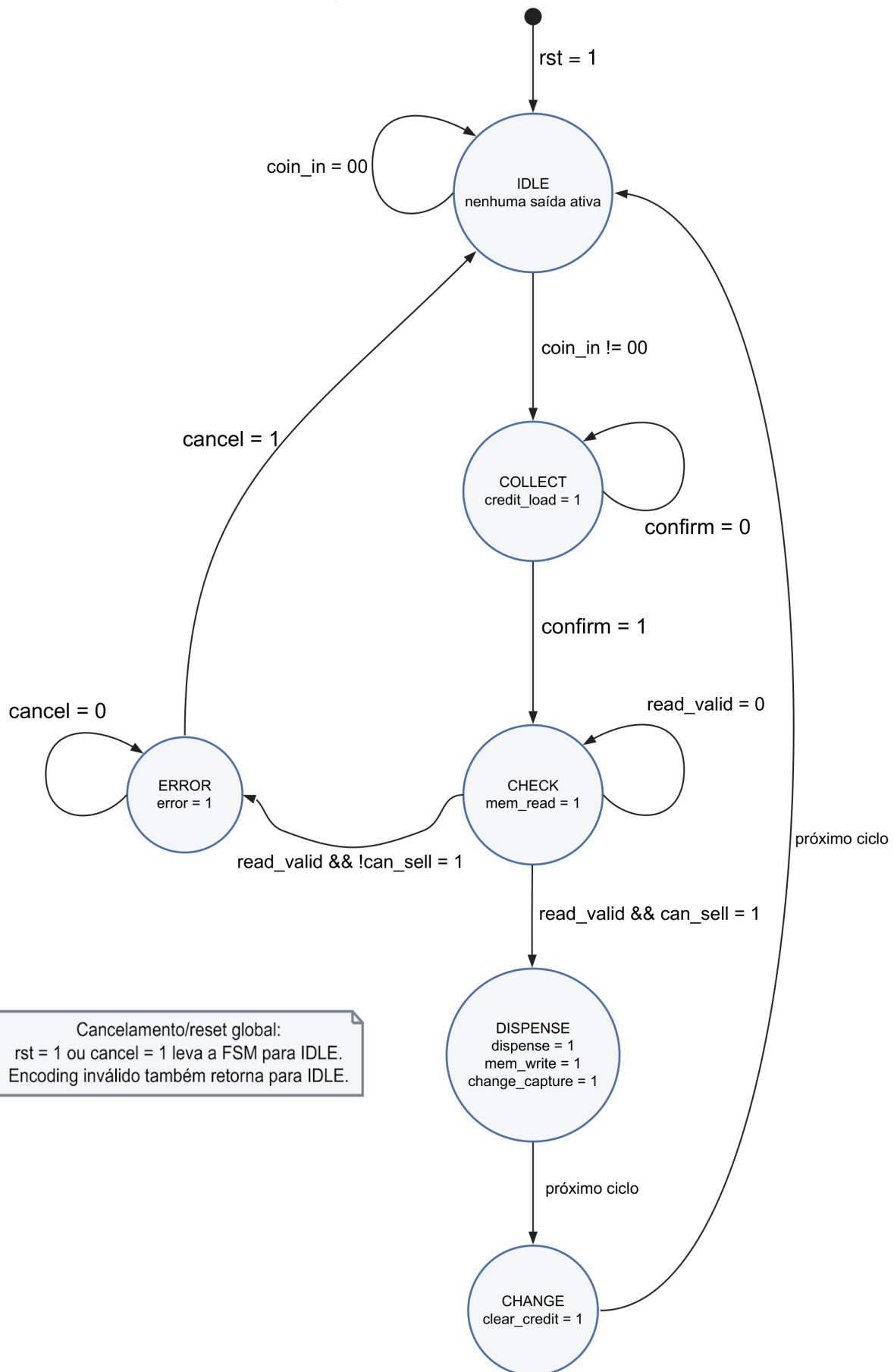


Figura 2: Diagrama de transição de estados e saídas de Moore.

4. Descrição dos módulos

Módulo	Decisão de projeto e justificativa
vending_pkg.v	Centraliza os códigos dos seis estados e a função <code>coin_to_value</code> . Dessa forma, os valores de R\$0,25, R\$0,50 e R\$1,00 são utilizados de modo consistente em todo o projeto.
credit_reg.v	Implementa um registrador síncrono de 8 bits. A prioridade é <code>rst > cancel > limpeza em CHANGE > acumulação</code> , garantindo devolução e limpeza.
memory.v	Modela quatro palavras de 16 bits, com <code>price</code> nos oito bits mais significativos e <code>stock</code> nos menos significativos. A interface de leitura é registrada e <code>read_valid</code> explicita a latência de um ciclo.
comparator.v / subtractor.v	Mantém o caminho aritmético puramente combinacional: <code>can_sell=(credit>=price)&&(stock>0)</code> e <code>change=credit-price</code> . A FSM é responsável por decidir quando utilizar cada resultado.
control_unit.v	Usa um bloco <code>always_ff</code> para o estado e um bloco <code>always_comb</code> para o próximo estado e as saídas.
vending_top.v	Instancia e interconecta todos os módulos. O primeiro crédito é aceito na borda que provoca <code>IDLE→COLLECT</code> , evitando perder a primeira moeda. <code>change_out</code> é registrado na transição <code>DISPENSE→CHANGE</code> ou durante <code>cancel</code> .

Tabela 1: Organização do RTL e decisões principais.

5. Waveforms anotadas dos cenários de teste

As capturas de forma de onda foram extraídas do Verdi a partir do arquivo `waves.fsdb`. Foram utilizadas as janelas temporais impressas pelo próprio testbench: reset inicial de 0 a 25 ns, cenário 1 de 25 a 105 ns, cenário 2 de 105 a 180 ns, cenário 3 de 180 a 240 ns e cenário 4 de 240 a 625 ns. Essas janelas também aparecem na validação FAIL/PASS de cada cenário, realizada com a função `check` no terminal, como será visto na próxima seção.

Em todas as capturas, o sinal `clk` foi utilizado como referência temporal. As entradas observadas foram `rst`, `coin_in`, `sel_item`, `confirm` e `cancel`. As saídas analisadas foram `state_out`, `display`, `dispense`, `change_out` e `error`.

No reset inicial, observa-se o sinal `rst` ativo durante dois ciclos de `clock`, inicializando a FSM em `IDLE` e zerando o crédito, o troco e os sinais de controle.

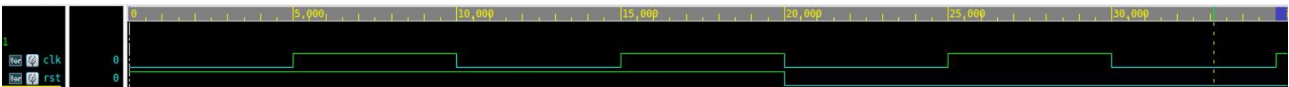


Figura 3: Waveform do reset inicial por dois ciclos de `clock`, mostrando a inicialização da FSM em `IDLE` e o crédito igual a zero.

No cenário 1, observa-se a inserção de R\$1,00 para a compra do café, a passagem pelos estados `DISPENSE` e `CHANGE`, a ativação de `dispense` por um ciclo e a geração de 75 centavos de troco.

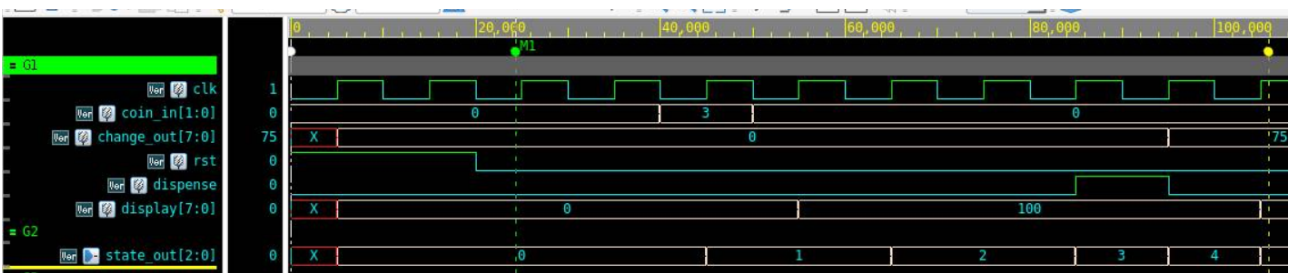
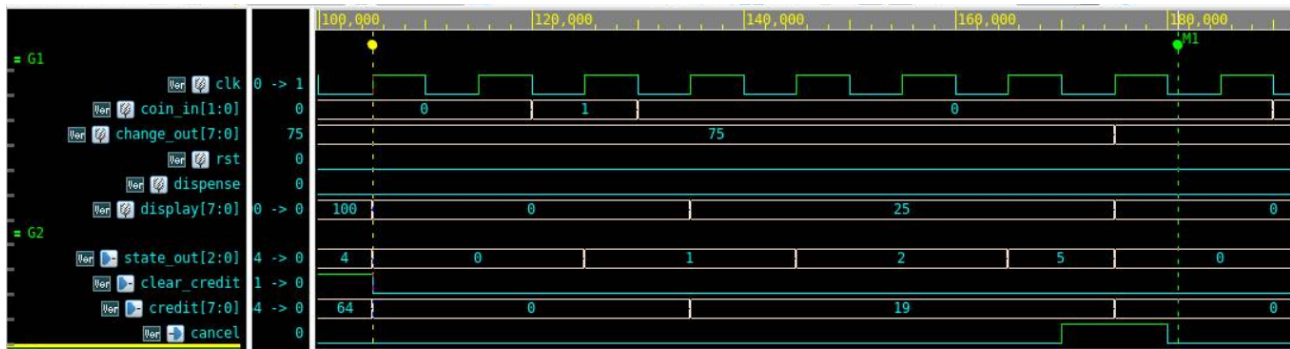


Figura 4: Cenário 1: compra bem-sucedida de café com R\$1,00 e geração de troco de 75 centavos.

No cenário 2, observa-se a tentativa de compra do snack com apenas 25 centavos, fazendo a FSM entrar em `ERROR` (`state_out = 5`). Após o acionamento de `cancel`, o valor inserido é devolvido e o sistema retorna para `IDLE`.



No cenário 3, observa-se o acúmulo de 200 centavos após a inserção de duas moedas de R\$1,00. Com o acionamento de **cancel**, o valor integral é transferido para **change_out** e o crédito é zerado.

No cenário 4, observam-se quatro novas vendas bem-sucedidas de café, com decremento progressivo do estoque e geração de 75 centavos de troco em cada operação. Na tentativa seguinte, o estoque zerado impede a venda e leva a FSM ao estado **ERROR**.

```

=====
[PASS] S2: FSM entra em ERROR | expected=5 actual=5
[PASS] S2: error permanece ativo | expected=1 actual=1
[PASS] S2: cancel retorna para IDLE | expected=0 actual=0
[PASS] S2: cancel devolve 25 centavos | expected=25 actual=25
[PASS] S2: credito zerado apos cancelamento | expected=0 actual=0
[TIME] Apos cenario 2: 180 ns

=====
CENARIO 3: cancelamento com R$2,00
=====
[PASS] S3: credito acumulado igual a 200 | expected=200 actual=200
[PASS] S3: FSM retorna para IDLE | expected=0 actual=0
[PASS] S3: cancel devolve 200 centavos | expected=200 actual=200
[PASS] S3: credito zerado apos cancelamento | expected=0 actual=0
[TIME] Apos cenario 3: 240 ns

=====
CENARIO 4: estoque de cafe zerado
=====
[PASS] S4: venda 2 entra em DISPENSE | expected=3 actual=3
[PASS] S4: venda 2 gera dispense | expected=1 actual=1
[PASS] S4: venda 2 entra em CHANGE | expected=4 actual=4
[PASS] S4: venda 2 devolve 75 centavos | expected=75 actual=75
[PASS] S4: venda 2 retorna para IDLE | expected=0 actual=0
[PASS] S4: venda 3 entra em DISPENSE | expected=3 actual=3
[PASS] S4: venda 3 gera dispense | expected=1 actual=1
[PASS] S4: venda 3 entra em CHANGE | expected=4 actual=4
[PASS] S4: venda 3 devolve 75 centavos | expected=75 actual=75
[PASS] S4: venda 3 retorna para IDLE | expected=0 actual=0
[PASS] S4: venda 4 entra em DISPENSE | expected=3 actual=3
[PASS] S4: venda 4 gera dispense | expected=1 actual=1
[PASS] S4: venda 4 entra em CHANGE | expected=4 actual=4
[PASS] S4: venda 4 devolve 75 centavos | expected=75 actual=75
[PASS] S4: venda 4 retorna para IDLE | expected=0 actual=0
[PASS] S4: venda 5 entra em DISPENSE | expected=3 actual=3
[PASS] S4: venda 5 gera dispense | expected=1 actual=1
[PASS] S4: venda 5 entra em CHANGE | expected=4 actual=4
[PASS] S4: venda 5 devolve 75 centavos | expected=75 actual=75
[PASS] S4: venda 5 retorna para IDLE | expected=0 actual=0
[PASS] S4: sexta tentativa entra em ERROR | expected=5 actual=5
[PASS] S4: error ativo pois o estoque esta zerado | expected=1 actual=1
[TIME] Apos cenario 4: 625 ns

=====
RESULTADO DO TESTBENCH: PASS (40 checks, 0 failures)
=====

```

A saída mostra 40 verificações e nenhuma falha. O cenário 4 aparece a partir da venda 2 porque a simulação foi executada de forma sequencial: o cenário 1 já consumiu uma unidade de café. Assim, as vendas 2 a 5 completam o esgotamento do estoque, e a tentativa seguinte corresponde à sexta compra total de café.

7. Resultados de síntese e exploração de constraints

A síntese foi executada no Synopsys Design Compiler (DC), utilizando a biblioteca de células-padrão `saed32rvt_tt1p05v25c.db`, fornecendo as células utilizadas no mapeamento do circuito, incluindo a célula `NBUFFX2_RVT`, empregada para modelar o acionamento das entradas. A biblioteca sintética `dw_foundation.sldb` também foi incluída.

O comando `check_design` foi executado antes da síntese e não indicou erros fatais. Ao final do processo, foram geradas a netlist sintetizada `vending_top_syn.v`, a representação `vending_top_syn.ddc` e os relatórios de área, timing, potência e violações de constraints.

As constraints foram definidas no arquivo `synth/vending.sdc`, incluindo clock, incerteza de clock, atrasos de entrada e saída, célula de acionamento das entradas e carga capacitiva nas saídas.

A análise inicial foi realizada com um período de clock de 20 ns, correspondente a uma frequência de 50 MHz. Nessa condição, o pior caminho de setup apresentou slack positivo de 15,43 ns, indicando que a constraint inicial foi atendida.

O pior caminho parte da entrada `coin_in[0]` e termina no registrador `u_credit_reg/credit_reg[7]`. O caminho atravessa a lógica de controle da inserção da moeda e a cadeia de somadores utilizada para atualizar o crédito acumulado.

Item	Resultado inicial
Período do clock	20 ns
Frequência do clock	50 MHz
Slack de setup	+15,43 ns
Resultado de setup	Atendido
Caminho crítico	coin_in[0] até u_credit_reg/credit_reg[7]
Número de células	280
Células combinacionais	206
Células sequenciais	62
Área combinacional	502,188546 u.a.
Área não combinacional	410,696717 u.a.
Área total de células	912,885263 u.a.
Área total	1080,284448 u.a.

Tabela 2: Resultados da síntese inicial com um período de 20 ns.

7.1 Exploração do período do clock

Para determinar a frequência máxima de operação, o período do clock foi reduzido progressivamente em passos de 2 ns. Para cada período, o projeto foi novamente sintetizado, registrando-se o pior slack de setup apresentado pelo comando `report_timing` e a área total apresentada pelo comando `report_area`.

A frequência correspondente a cada período foi calculada por:

$$f = \frac{1000}{T}, \quad (1)$$

em que T é o período em nanossegundos e f é a frequência em megahertz.

Período (ns)	Frequência (MHz)	Slack de setup (ns)	Área total (u.a.)	Resultado
20	50,00	+15,43	1080,284448	Atendido
18	55,56	+13,43	1080,284448	Atendido
16	62,50	+11,43	1080,284448	Atendido
14	71,43	+9,43	1080,284448	Atendido
12	83,33	+7,43	1080,284448	Atendido
10	100,00	+5,43	1080,284448	Atendido
8	125,00	+3,42	1083,566170	Atendido
6	166,67	+1,43	1085,960390	Atendido
4	250,00	-0,14	1427,170008	Violado

Tabela 3: Exploração do período do clock, do slack de setup e da área total.

O menor período para o qual o slack permaneceu não negativo foi de 6 ns, correspondente a uma frequência de aproximadamente 166,67 MHz. Nesse ponto, o slack de setup foi de +1,43 ns e a área total sintetizada foi de 1085,960390 u.a.

Ao reduzir o período para 4 ns, isto é, 2 ns abaixo do menor período atendido, o DC apresentou slack de -0,14 ns. Dessa forma, o timing passou a ser violado. A área total nessa condição foi de 1427,170008 u.a., indicando que o sintetizador aumentou a área na tentativa de atender à constraint mais restritiva.

Entre os períodos de 20 ns e 10 ns, a área total permaneceu constante. Nos períodos de 8 ns e 6 ns, foram observados pequenos aumentos de área. Entretanto, para o período de 4 ns, a área aumentou aproximadamente 31,42% em relação à síntese de 6 ns. O número total de células também aumentou de 283 para 431, mostrando que o DC realizou um mapeamento mais agressivo na tentativa de reduzir os atrasos.

Consequentemente, a frequência máxima obtida para o projeto, considerando a exploração em passos de 2 ns, foi:

$$f_{\max} = \frac{1000}{T_{\min}} = \frac{1000}{6} = 166,67 \text{ MHz}. \quad (2)$$

7.2 Comparação da opção `-no_autoungroup`

Após encontrar o menor período com slack não negativo, a síntese foi repetida retirando-se a opção `-no_autoungroup` do comando `compile_ultra`. Essa comparação permite verificar se a eliminação automática da hierarquia produz alguma alteração na área e no desempenho temporal.

Configuração	Período (ns)	Slack (ns)	Área total (u.a.)	Células
Com <code>-no_autoungroup</code>	6	+1,43	1085,960390	283
Sem <code>-no_autoungroup</code>	6	+1,39	1033,349343	246

Tabela 4: Comparação da síntese com e sem `-no_autoungroup`.

A retirada da opção `-no_autoungroup` não alterou significativamente o desempenho temporal. O slack de setup foi reduzido de +1,43 ns para +1,39 ns, uma diferença de apenas 0,04 ns, mantendo o timing atendido para o período de 6 ns.

Por outro lado, a área total foi reduzida de 1085,960390 u.a. para 1033,349343 u.a., correspondendo a uma redução de aproximadamente 4,84%. O número total de células também diminuiu de 283 para 246. A quantidade de células combinacionais foi reduzida de 220 para 183, enquanto o número de células sequenciais permaneceu igual a 62.

Esse resultado ocorre porque, sem a preservação explícita da hierarquia, o DC pode realizar otimizações entre módulos, eliminar limites hierárquicos, compartilhar expressões equivalentes e remover lógica redundante. Neste projeto, essas otimizações reduziram a área sem comprometer o fechamento de timing para o menor período atendido.

8. Análise do caminho crítico

Para o menor período atendido na exploração principal, de 6 ns, o pior caminho de setup parte da entrada `coin_in[1]` e termina no registrador `u_credit_reg/credit_reg[7]`. O caminho apresenta tempo de chegada de 4,05 ns, tempo requerido de 5,48 ns e slack positivo de 1,43 ns. Esse caminho atravessa principalmente o módulo `credit_reg`. Após a entrada `coin_in[1]`, o sinal percorre a lógica de detecção e controle da inserção da moeda e a cadeia aritmética utilizada para somar o valor da moeda ao crédito acumulado.

Ao reduzir o período para 4 ns, o DC realizou um novo mapeamento e o pior caminho passou a partir da entrada `sel_item[0]` e terminar no registrador `u_memory/mem_reg[3]` [3]. Esse caminho atravessa o módulo `vending_memory`, incluindo a lógica de decodificação do endereço e de atualização da posição selecionada da memória. A mudança do caminho crítico mostra que o sintetizador conseguiu otimizar parcialmente o caminho de acumulação do crédito.

9. Equivalência Formal (Formality) com guidance de SVF

A verificação de equivalência formal foi realizada com o Synopsys Formality, utilizando netlists obtidas em sínteses com período de *clock* de 6 ns. Os arquivos RTL originais do controlador de `vending machine` foram adotados como design de referência (*golden*), enquanto as netlists geradas pelo Synopsys Design Compiler foram adotadas como designs revisados (*revision*), com o módulo `vending_top` definido como módulo superior. Foram realizadas duas rodadas: uma com `compile_ultra -no_autoungroup` e outra com `compile_ultra`. Em cada rodada, o arquivo SVF foi gerado com a respectiva netlist e utilizado como *guidance* para o casamento dos pontos de comparação. Os relatórios da primeira rodada foram salvos antes da segunda síntese, que regenerou a netlist e o arquivo SVF.

O arquivo `fm/formality.tcl` foi desenvolvido manualmente e utilizado nas duas verificações, enquanto o arquivo `fm/formality_auto.tcl`, gerado pelo utilitário `fm_mk_script`, foi utilizado apenas para comparação. Esta seção segue os oito itens estabelecidos na Seção 15 do roteiro. As Subseções 9.1 a 9.5 apresentam os resultados da netlist sintetizada com `-no_autoungroup`, e a Subseção 9.6 compara os resultados obtidos com e sem essa opção.

9.1 Geração do arquivo SVF durante a síntese

O arquivo SVF foi gerado pelo Design Compiler durante a mesma execução que produziu a netlist `synth/vending_top_syn.v`. Para isso, o comando `set_svf` foi inserido no arquivo `synth/synth.tcl` imediatamente antes de `compile_ultra`, conforme apresentado a seguir.

```
redirect $SYNTH_DIR/reports/check_design.rpt {
  check_design
}

# Arquivo SVF para verificacao formal
set_svf $FORMAL_DIR/reports/default.svf

compile_ultra -no_autoungroup
#compile_ultra
```

Como `FORMAL_DIR` foi definido como `$ROOT_DIR/fm`, o arquivo foi armazenado em `fm/reports/default.svf`. Em cada rodada, o arquivo SVF foi regenerado juntamente com a respectiva netlist, evitando a utilização de um SVF correspondente a uma execução de síntese diferente.

```
[apolo.silva@srv-microeletronica3 grupo_NN_vending]$ ls -lh fm/reports/default.svf
-rw-rw-r--. 1 apolo.silva apolo.silva 12K Jul 29 14:55 fm/reports/default.svf
```

Figura 8: Confirmação da geração do arquivo `default.svf`.

9.2 Configuração do SVF no Formality

O arquivo `fm/formality.tcl` habilita o modo de configuração automática, carrega o SVF e, em seguida, lê o design de referência e o design revisado. Embora o roteiro apresente o comando `set_svf` antes da configuração da variável `synopsys_auto_setup`, essa ordem produziu o erro apresentado na Figura 9. A mensagem informa que `synopsys_auto_setup` não pode ser alterada após a execução de `set_svf`.

```
# 3. Guidance
set_svf $FORMAL_DIR/reports/default.svf; # gerado a partir da síntese em dc
SVF set to '/home/apolosilva/atividades_roteiros/vending_machine/formal/grupo_NN_vending/fm/reports/default.svf'.
1
# 2. habilita o modo de setup automático baseado no SVF
set synopsys_auto_setup true
Error: can't set "synopsys_auto_setup": Synopsys_auto_setup may not be changed after set_svf command is issued
Use error_info for more info. (CMD-013)
Information: script 'fm/formality.tcl'
stopped at line 28 due to error. (CMD-081)
Extended error info:
Synopsys_auto_setup may not be changed after set_svf command is issued
(write trace on "synopsys_auto_setup")
invoked from within
"set synopsys_auto_setup true"
(file "fm/formality.tcl" line 28)
-- End Extended Error Info
```

Figura 9: Erro causado pela configuração de `synopsys_auto_setup` após a execução de `set_svf`.

Diante disso, a variável `synopsys_auto_setup` foi habilitada antes da leitura do arquivo SVF. Essa mesma ordem é recomendada no material do curso remoto. Após essa correção, o arquivo `fm/formality.tcl` foi executado até o fim sem erros fatais.

O trecho correto utilizado no arquivo `fm/formality.tcl` é apresentado a seguir.

```
set TOP vending_top; # top do design a verificar

# Habilita o modo de setup automatico baseado no SVF
set synopsys_auto_setup true

# Guidance gerado durante a síntese
set_svf $FORMAL_DIR/reports/default.svf

# Design de referencia (golden): RTL pre-síntese
read_sverilog -r $rtl_files
set_top r:/WORK/$TOP

# Design revisado: netlist do Design Compiler
read_db $LIBS_DIR/saed32rvttt1p05v25c.db
read_verilog -i $SYNTH_DIR/${TOP}_syn.v
set_top i:/WORK/$TOP
```

O design de referência foi formado pelos arquivos `vending_pkg.sv`, `credit_reg.sv`, `memory.sv`, `comparator.sv`, `subtractor.sv`, `control_unit.sv` e `vending_top.sv`, cujos caminhos foram armazenados na variável `rtl_files` e carregados pelo comando `read_sverilog`. Para o design revisado, foram carregadas a biblioteca `saed32rvttt1p05v25c.db` e a netlist `vending_top_syn.v`.

O arquivo `formality_auto.tcl` foi gerado por meio do comando:

```
fm_mk_script -o fm/formality_auto.tcl "fm/reports/default.svf"
```

Esse arquivo foi utilizado apenas para conferir o script manual. O script automático recuperou o modo Synopsys Auto Setup, os caminhos de busca, o arquivo SVF e a biblioteca tecnológica. Entretanto, ele não recuperou do SVF os comandos de leitura do RTL, registrando a mensagem `No guide_environment analyze commands found in SVF`. Desse modo, os comandos de leitura dos designs deveriam ser inseridos manualmente. Por esse motivo, optou-se por executar a verificação com o arquivo `formality.tcl` escrito manualmente.

9.3 Operações do SVF aceitas e rejeitadas

Após o comando `match`, foram gerados os relatórios das operações do SVF aceitas e rejeitadas:

```
report_svf_operation -status accepted \
> $FORMAL_DIR/reports/formality_svf_accepted.rpt

report_svf_operation -status rejected \
> $FORMAL_DIR/reports/formality_svf_rejected.rpt
```

Como o relatório de operações aceitas é extenso, a Figura 10 apresenta apenas seu trecho inicial, enquanto a Tabela 5 sintetiza todas as operações processadas. O relatório de operações rejeitadas é apresentado integralmente na Figura 11. Nenhuma operação foi rejeitada.


```

1 *****
2 Report      : svf_operation
3             -status accepted
4
5 Reference   : r:/WORK/vending_top
6 Implementation : i:/WORK/vending_top
7 Version     : X-2025.06-SP3
8 Date       : Wed Jul 29 01:52:11 2026
9 *****
10
11 ## SVF Operation 1 (Line: 21) - environment. Status: accepted
12 ## Operation Id: 1

```

Figura 10: Trecho inicial do relatório das operações do SVF aceitas pelo Formality.

```

1 *****
2 Report      : svf_operation
3             -status rejected
4
5 Reference   : r:/WORK/vending_top
6 Implementation : i:/WORK/vending_top
7 Version     : X-2025.06-SP3
8 Date       : Wed Jul 29 01:52:11 2026
9 *****
10 1

```

Figura 11: Relatório das operações do SVF rejeitadas pelo Formality.

Na verificação da netlist sintetizada com `-no_autoungroup`, o Formality processou 70 operações registradas no SVF. Todas foram aceitas, sem operações rejeitadas, pendentes ou não processadas (Tabela 5).

Operação	Aceitas	Rejeitadas	Pendentes	Total
architecture_netlist	1	0	0	1
boundary	1	0	0	1
boundary_netlist	1	0	0	1
change_names	7	0	0	7
datapath	1	0	0	1
environment	4	0	0	4
instance_map	1	0	0	1
merge	2	0	0	2
reg_constant	49	0	0	49
replace	2	0	0	2
ungroup	1	0	0	1
Total	70	0	0	70

Tabela 5: Operações de `guidance` processadas pelo Formality para a netlist sintetizada com `-no_autoungroup`.

As operações mais numerosas foram as 49 operações `reg_constant`, associadas principalmente à substituição de bits constantes dos registradores da memória. Também foram aceitas sete operações `change_names`, duas operações `merge`, duas operações `replace` e uma operação `ungroup`. O relatório `formality_svf_rejected.rpt` apresentou somente o cabeçalho, sem listar qualquer operação rejeitada.

9.4 Pontos não casados

O casamento entre os designs foi executado com o comando `match`. Na verificação da netlist principal, o log do Formality apresentou os seguintes resultados:

```

83 Compare points matched by name
0 Compare points matched by signature analysis
0 Compare points matched by topology
8 Matched primary inputs, black-box outputs
0(0) Unmatched reference(implementation) compare points
0(0) Unmatched reference(implementation) primary inputs, black-box outputs

```

O relatório `formality_matched.rpt` contabilizou 91 pontos casados. Esse total corresponde aos 83 pontos de comparação e às oito entradas primárias também casadas pelo Formality. O arquivo `formality_unmatched.rpt` apresentou:

```
No unmatched points.
```

Assim, não restaram pontos sem correspondência entre o design de referência e o design revisado, nem pontos classificados como NOT COMPARED.

9.5 Resultado da equivalência da netlist principal

Após o casamento dos pontos, a prova de equivalência foi executada com o comando `verify`. A verificação da netlist sintetizada com `-no_autoungroup` foi concluída com o veredito **Verification SUCCEEDED**. Todos os 83 pontos de comparação foram provados equivalentes, sendo 21 portas de saída e 62 registradores. Não foram identificados pontos de comparação em falha. O veredito final foi obtido por meio do comando `report_status`. A opção `-verbose`, indicada no roteiro, não foi utilizada, pois não consta entre as opções aceitas pelo comando na versão do Formality disponível no ambiente de execução, resultando no erro **unknown option '-verbose'** (CMD-010). Dessa forma, utilizou-se o comando `report_status` sem argumentos adicionais.

```

32 ***** Verification Results *****
33 Verification SUCCEEDED
34   ATTENTION: synopsys_auto_setup mode was enabled.
35             See Synopsys Auto Setup Summary for details.
36   ATTENTION: Inconsistency detected in design data with SVF.
37             See Design Consistency Check Summary for details.
38 -----
39 Reference design: r:/WORK/vending_top
40 Implementation design: i:/WORK/vending_top
41 83 Passing compare points
42 -----
43 Matched Compare Points      BBPin   Loop   BBNet   Cut   Port   DFF   LAT   TOTAL
44 -----
45 Passing (equivalent)         0       0       0       0    21    62     0    83
46 Failing (not equivalent)     0       0       0       0     0     0     0     0
47 *****

```

Figura 12: Resultado do comando `report_status` para a verificação de equivalência da netlist principal.

O log também registrou a advertência FM-622, indicando a ausência de `guide_file_info` para os sete arquivos RTL. Apesar dessa advertência, todas as 70 operações do SVF foram aceitas, não houve pontos não casados e a prova de equivalência foi concluída com sucesso.

9.6 Comparação com e sem `-no_autoungroup`

A segunda verificação foi realizada com a netlist produzida por `compile_ultra`, sem a opção `-no_autoungroup`. Essa execução também foi concluída com o veredito **Verification SUCCEEDED**, apresentando 83 pontos de comparação equivalentes, dos quais 21 correspondem a portas de saída e 62 a registradores.

A comparação dos resultados obtidos nas duas execuções é apresentada na Tabela 6.

Resultado	Com <code>-no_autoungroup</code>	Sem <code>-no_autoungroup</code>
Período de <i>clock</i>	6 ns	6 ns
Comando de síntese	<code>compile_ultra</code> <code>-no_autoungroup</code>	<code>compile_ultra</code>
Veredito final	SUCCEEDED	SUCCEEDED
Compare points equivalentes	83	83
Pontos casados no total	91	91
Pontos não casados	0	0
Operações SVF aceitas	70	77
Operações SVF rejeitadas	0	0
Tempo de processamento (CPU)	2,82 s	2,50 s
Tempo total de execução	7 s	7 s

Tabela 6: Comparação dos resultados de equivalência formal obtidos com e sem a opção `-no_autoungroup`.

A retirada da opção `-no_autoungroup` não alterou o resultado da prova de equivalência. Nas duas configurações, todos os 83 pontos de comparação foram provados equivalentes e não foram identificados pontos divergentes ou não casados. O tempo total de execução foi de 7 s em ambas as rodadas. Na execução sem `-no_autoungroup`, o tempo de processamento foi reduzido de 2,82 s para 2,50 s.

A principal diferença ocorreu no número de operações do SVF. A síntese sem `-no_autoungroup` produziu 77 operações aceitas, contra 70 na síntese com a opção. O número de operações `ungroup` aumentou de uma para seis, o número de operações `environment` aumentou de quatro para cinco e foi adicionada uma operação `inv_push`. Essas sete operações adicionais são compatíveis com as transformações hierárquicas e lógicas permitidas na segunda síntese.

9.7 Análise de eventuais pontos divergentes

Na verificação da netlist sintetizada com `-no_autoungroup`, o arquivo `formality_failing.rpt` apresentou:

```
No failing compare points.
```

O relatório correspondente à netlist sintetizada sem `-no_autoungroup` apresentou o mesmo resultado. Consequentemente, não foi necessário executar `analyze_points`, rastrear cones lógicos divergentes ou propor alterações no RTL, no script de síntese ou no arquivo SDC.

9.8 Discussão sobre o uso do guidance de SVF

O arquivo SVF forneceu ao Formality o histórico das transformações realizadas pelo Design Compiler. Na execução com `-no_autoungroup`, todas as 70 operações registradas foram aceitas. Na execução sem essa opção, todas as 77 operações também foram aceitas. Em nenhuma das duas verificações foi necessário utilizar comandos manuais adicionais de casamento, como `match -certain` ou `set_user_match`.

A síntese sem `-no_autoungroup` gerou mais operações relacionadas à modificação da hierarquia, especialmente seis operações `ungroup`, em comparação com apenas uma na primeira rodada. Apesar dessas transformações adicionais, o Formality casou os mesmos 83 pontos de comparação por nome e provou a equivalência das duas netlists em relação ao RTL original.

O arquivo `formality_auto.tcl` não foi utilizado para substituir o fluxo manual. Sua função foi apenas comparar as configurações inferidas pelo `fm_mk_script` com aquelas implementadas no arquivo `formality.tcl`.

10. Conclusão

O projeto implementou um controlador de vending machine composto por uma FSM de Moore, memória síncrona, registrador acumulador de crédito e blocos combinacionais de comparação e subtração. Os quatro cenários obrigatórios foram executados por um testbench self-checking, resultando em 40 verificações aprovadas e nenhuma falha.

As principais dificuldades estiveram relacionadas ao alinhamento temporal dos sinais de entrada e dos sinais síncronos. Foi necessário garantir que a primeira moeda fosse amostrada corretamente pela FSM e pelo registrador de crédito, além de considerar a latência de um ciclo da memória antes da decisão de venda. A execução sequencial dos cenários também precisou ser considerada, pois a compra realizada no primeiro cenário alterou o estoque utilizado posteriormente no teste de estoque zerado.

Na síntese inicial, realizada com um período de 20 ns, o circuito apresentou slack de setup positivo de 15,43 ns e área total de 1080,28448 u.a. O período foi reduzido em passos de 2 ns, conforme solicitado, até que fosse observada uma violação. O menor período atendido foi de 6 ns, com slack positivo de 1,43 ns e área total de 1085,960390 u.a. Dessa forma, a maior frequência verificada pela exploração foi de aproximadamente 166,67 MHz.

Para o período de 4 ns, correspondente a 250 MHz, o slack tornou-se negativo, atingindo -0,14 ns. Nessa condição, o DC aumentou a área total para 1427,170008 u.a. e o número de células para 431, mas não conseguiu fechar o timing. O caminho crítico também mudou do acumulador de crédito para a lógica de seleção e atualização da memória.

A comparação realizada sem a opção `-no_autoungroup`, no período de 6 ns, apresentou slack de 1,39 ns e área total de 1033,349343 u.a. A remoção da opção reduziu a área em aproximadamente 4,84%, mantendo o timing atendido e provocando apenas uma pequena redução de 0,04 ns no slack.

Na primeira rodada de verificação formal, correspondente à netlist sintetizada com `-no_autoungroup`, o Formality aceitou 70 operações do SVF, não rejeitou nenhuma operação e não deixou pontos não casados. A prova terminou com o veredito `SUCCEEDED`, totalizando 83 pontos de comparação equivalentes, dos quais 21 correspondem a portas de saída e 62 a registradores. Na segunda rodada, referente à netlist sintetizada sem `-no_autoungroup`, a equivalência também foi comprovada, confirmando que ambas as estratégias de síntese preservaram o comportamento funcional do RTL.

De forma geral, as atividades permitiram colocar em prática os conceitos apresentados nos laboratórios e no curso remoto.